LATIHAN TERSTRUKTUR (TUGAS KULIAH)

Pertemuan 4: Implementasi Unit Testing dengan GitHub Actions

Nama : **Nadjlah Cahya Nerik Fadilah**
NIM : **3224001**
Kelas : **Sistem Informasi**

BAGIAN A: GIT & GITHUB DASAR (25 POIN)

Soal 1 (10 Poin)

Jelaskan dengan singkat dan jelas:

1. Apa perbedaan antara **Centralized VCS** dan **Distributed VCS**? Berikan masing-masing 2 contoh!

Jawaban:

Centralized VCS adalah sistem version control yang menggunakan satu server pusat sebagai tempat penyimpanan utama repository. Jadi pengguna harus terhubung ke server tersebut untuk mendapatkan atau mengirim perubahan.

Contoh:

- SVN (Subversion)
- CVS (Concurrent Versions System)

Distributed VCS adalah sistem version control yang setiap pengguna mempunyai salinan repository secara lengkap di komputer lokal. Jadi proses commit bisa dilakukan secara lokal terlebih dahulu dan perubahan dapat dikirim ke repository lain.

Contoh:

- Git
- Mercurial

Perbedaannya:

```
Centralized VCS bergantung pada satu server pusat, sedangkan
Distributed VCS mempunyai repository lokal pada masing-masing pengguna.
```

2. Sebutkan 5 **Git commands** yang wajib dikuasai beserta fungsinya!

   Jawaban:

Git Command	Fungsi
`git init`	Membuat repository Git baru di folder
`git clone`	Menyalin repository dari remote ke komputer lokal
`git add`	Menambahkan file ke staging area
`git commit`	Menyimpan perubahan ke repository lokal
`git push`	Mengirim perubahan dari lokal ke repository remote

3. Apa fungsi **.gitignore** dalam repository Git?

   `Jawaban :`

   `gitignore` digunakan untuk menentukan file atau folder yang tidak ingin ikut dilacak oleh Git. Biasanya digunakan untuk file sementara, file konfigurasi pribadi, hasil build, atau file yang tidak perlu dimasukkan ke repository.

   Contohnya:

```
node_modules/
.env
target/
*.log
```

Soal 2 (15 Poin)

Praktik Git Commands:

Lakukan langkah-langkah berikut dan tuliskan **command** yang Anda gunakan:

1. Buat repository baru di GitHub dengan nama `latihan-git-[NIM]`
2. Clone repository tersebut ke lokal
3. Buat file `README.md` dan tuliskan nama serta NIM Anda
4. Commit file tersebut dengan pesan **"docs: add README"**
5. Push ke GitHub

`feature-test`

6. Buat branch baru dengan nama
7. Pindah ke branch tersebut
8. Buat file `test.txt` dan commit dengan pesan **"test: add test file"**

9. Push branch `feature-test` ke GitHub
10. Merge branch `feature-test` ke `main`

Tuliskan semua command yang digunakan!

Jawaban:

Buat repository baru di GitHub dengan nama latihan-git-3224001 melalui https://github.com/new, kemudian jalankan command berikut di terminal:

```
git clone https://github.com/nazlaej/latihan-git-3224001.git
cd latihan-git-3224001
echo Nadjlah Cahya Nerik Fadilah - 3224001 > README.md
```

Commit lalu push ke GitHub (langkah 4 dan 5):

```
git add README.md
git commit -m "docs: add README"
git push origin main
```

Buat branch baru feature-test dan pindah ke branch tersebut (langkah 6 dan 7):

```
git checkout -b feature-test
```

Buat file test.txt (langkah 8), kemudian commit dan push branch feature-test (langkah 9):

```
echo test > test.txt
git add test.txt
git commit -m "test: add test file"
git push origin feature-test
```

Kembali ke main, merge branch feature-test, lalu push (langkah 10):

```
git checkout main
git merge feature-test
git push origin main
```

Untuk verifikasi, dapat digunakan: git status, git log --oneline --all, dan git branch -a.

BAGIAN B: UNIT TESTING FRAMEWORK (35 POIN)

Soal 3 (15 Poin)

Konversi Test Case ke Kode Unit Test:

Berikut adalah kode program dari **Tugas Minggu 3** (Soal 3 - Validasi Pendaftaran):

```
java
public String validasiPendaftaran(String nama, int umur, String email, boolean setujuSyarat) {
    if (nama == null || nama.isEmpty()) {              // P1
        return "Nama harus diisi";
    }

    if (umur < 17) {                                   // P2
        return "Umur minimal 17 tahun";
```

```
    }

    if (email == null || !email.contains("@")) {       // P3, P4
        return "Email tidak valid";
    }

    if (!setujuSyarat) {                               // P5
        return "Harus menyetujui syarat dan ketentuan";
    }

    return "Pendaftaran berhasil";
}
```

Tugas:

1. Tuliskan **5 test case** berdasarkan independent path yang telah Anda buat di Minggu 3! (5 poin)
2. Implementasikan test case tersebut ke dalam **kode unit test JUnit**! (10 poin)

Jawaban a. 5 test case berdasarkan independent path (P1 s.d. P5) dari Minggu 3:
1. Nama kosong - input: nama = "" (atau null), umur = 20, email = "a@mail.com", setuju = true - hasil: "Nama harus diisi" (jalur P1).
2. Umur di bawah 17 - input: nama = "Nadjlah", umur = 16, email = "a@mail.com", setuju = true - hasil: "Umur minimal 17 tahun" (jalur P1 false, P2 true).
3. Email tidak mengandung @ - input: nama = "Nadjlah", umur = 20, email = "nadjlahmail.com", setuju = true - hasil: "Email tidak valid" (jalur P1 false, P2 false, P3/P4 true).
4. Tidak menyetujui syarat - input: nama = "Nadjlah", umur = 20, email = "a@mail.com", setuju = false - hasil: "Harus menyetujui syarat dan ketentuan" (jalur P1 false, P2 false, P3 false, P5 true).
5. Semua valid - input: nama = "Nadjlah", umur = 20, email = "nadjlah@mail.com", setuju = true - hasil: "Pendaftaran berhasil" (jalur sukses).

Jawaban b. Implementasi unit test dengan JUnit 5:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ValidasiPendaftaranTest {

    private final ValidasiPendaftaran v = new ValidasiPendaftaran();

    @Test
    public void testNamaKosong() {
        assertEquals("Nama harus diisi", v.validasiPendaftaran("", 20, "a@mail.com",
true));
        assertEquals("Nama harus diisi", v.validasiPendaftaran(null, 20, "a@mail.com",
true));
    }

    @Test
    public void testUmurKurangDari17() {
        assertEquals("Umur minimal 17 tahun",
                v.validasiPendaftaran("Nadjlah", 16, "a@mail.com", true));
    }

    @Test
    public void testEmailTidakValid() {
        assertEquals("Email tidak valid",
                v.validasiPendaftaran("Nadjlah", 20, "nadjlahmail.com", true));
        assertEquals("Email tidak valid",
                v.validasiPendaftaran("Nadjlah", 20, null, true));
    }

    @Test
    public void testTidakMenyetujuiSyarat() {
        assertEquals("Harus menyetujui syarat dan ketentuan",
                v.validasiPendaftaran("Nadjlah", 20, "a@mail.com", false));
    }

    @Test
    public void testPendaftaranBerhasil() {
        assertEquals("Pendaftaran berhasil",
                v.validasiPendaftaran("Nadjlah", 20, "nadjlah@mail.com", true));
    }
}
```

Soal 4 (20 Poin)

Implementasi Unit Test untuk Kode Berikut:

java

```java
public class StudentGrade {

    public String calculateGrade(int score) {
        if (score < 0 || score > 100) {
            throw new IllegalArgumentException("Score must be between 0 and 100");
        }

        if (score >= 80) {
            return "A";
        } else if (score >= 70) {
            return "B";
        } else if (score >= 60) {
            return "C";
        } else if (score >= 50) {
            return "D";
        } else {
            return "E";
        }
    }

    public boolean isPassed(int score) {
        if (score < 0 || score > 100) {
            throw new IllegalArgumentException("Score must be between 0 and 100");
        }
        return score >= 60;
    }
}
```

Tugas:

1. Hitung **Cyclomatic Complexity** dari method `calculateGrade()`! (3 poin)
2. Tentukan **Independent Path** dari method `calculateGrade()`! (5 poin)
3. Implementasikan **unit test JUnit** untuk kedua method (`calculateGrade` dan

Jawaban a. Cyclomatic Complexity method calculateGrade():
Rumus: V(G) = E - N + 2P, atau sederhana V(G) = jumlah decision point + 1 (P = 1 graph terhubung).
Decision point dalam calculateGrade(): (1) score < 0, (2) score > 100 pada operator ||, (3) score >= 80, (4) score >= 70, (5) score >= 60, (6) score >= 50, sehingga berjumlah 6.
Maka V(G) = 6 + 1 = 7. Dengan E = 19 edge dan N = 14 node: V(G) = 19 - 14 + 2 = 7. Jadi Cyclomatic Complexity = 7.

Jawaban b. Independent path (7 path):
Path 1: score < 0 -> throw IllegalArgumentException.
Path 2: score > 100 -> throw IllegalArgumentException.
Path 3: 80 <= score <= 100 -> return "A".
Path 4: 70 <= score < 80 -> return "B".
Path 5: 60 <= score < 70 -> return "C".
Path 6: 50 <= score < 60 -> return "D".
Path 7: 0 <= score < 50 -> return "E".

Jawaban c. Unit test JUnit 5 untuk method calculateGrade() dan isPassed():

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class StudentGradeTest {

    private final StudentGrade sg = new StudentGrade();

    @Test
    public void testGradeA() {
        assertEquals("A", sg.calculateGrade(80));
        assertEquals("A", sg.calculateGrade(100));
    }

    @Test
    public void testGradeB() {
        assertEquals("B", sg.calculateGrade(70));
        assertEquals("B", sg.calculateGrade(79));
    }

    @Test
    public void testGradeC() {
        assertEquals("C", sg.calculateGrade(60));
        assertEquals("C", sg.calculateGrade(69));
    }

    @Test
    public void testGradeD() {
        assertEquals("D", sg.calculateGrade(50));
        assertEquals("D", sg.calculateGrade(59));
    }

    @Test
    public void testGradeE() {
        assertEquals("E", sg.calculateGrade(0));
```

```
        assertEquals("E", sg.calculateGrade(49));
    }

    @Test
    public void testScoreNegatif() {
        assertThrows(IllegalArgumentException.class, () -> sg.calculateGrade(-1));
    }

    @Test
    public void testScoreLebihDari100() {
        assertThrows(IllegalArgumentException.class, () -> sg.calculateGrade(101));
    }

    @Test
    public void testIsPassedLulus() {
        assertTrue(sg.isPassed(60));
        assertTrue(sg.isPassed(100));
    }

    @Test
    public void testIsPassedTidakLulus() {
        assertFalse(sg.isPassed(59));
        assertFalse(sg.isPassed(0));
    }

    @Test
    public void testIsPassedNilaiTidakValid() {
        assertThrows(IllegalArgumentException.class, () -> sg.isPassed(-1));
        assertThrows(IllegalArgumentException.class, () -> sg.isPassed(101));
    }
}
```

`isPassed`)! (12 poin)

BAGIAN C: GITHUB ACTIONS (25 POIN)

Soal 5 (15 Poin)

Analisis Workflow YAML:

Perhatikan workflow GitHub Actions berikut:

```yaml
name: Python CI

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
```

```yaml
jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        python-version: ['3.10', '3.11', '3.12']

    steps:
      - name: Checkout code
        uses: actions/checkout@v4
```

```yaml
      - name: Setup Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install pytest pytest-cov

      - name: Run tests
        run: pytest --cov=src --cov-report=xml
```

Tugas:

1. Apa **nama workflow** di atas? (2 poin)
2. **Trigger/event** apa saja yang memicu workflow? (3 poin)
3. Apa fungsi dari **strategy matrix** pada workflow di atas? (5 poin)
4. Sebutkan **5 steps** yang ada di dalam job `test`! (5 poin)

Jawaban:
1. Nama workflow: "Python CI".
2. Trigger/event: push ke branch main dan develop, serta pull request ke branch main.
3. Fungsi strategy matrix: menjalankan job "test" yang sama secara paralel untuk setiap nilai pada python-version (3.10, 3.11, dan 3.12), sehingga menghasilkan 3 job sekaligus. Fungsinya untuk memastikan kode tetap berjalan/kompatibel pada beberapa versi Python.
4. Steps yang ada di dalam job test:
   1) Checkout code - uses: actions/checkout@v4
   2) Setup Python sesuai matrix - uses: actions/setup-python@v5
   3) Install dependencies - upgrade pip, lalu install pytest dan pytest-cov
   4) Run tests - pytest --cov=src --cov-report=xml

Catatan: pada YAML yang diberikan pada soal hanya terdapat 4 steps; langkah ke-5 (misalnya upload coverage report) tidak ada pada potongan kode tersebut.

Soal 6 (10 Poin)

Membuat Workflow YAML:

Buatlah **workflow GitHub Actions** untuk project **Java** dengan ketentuan:

- Nama workflow: **"Java Unit Test"**
- Trigger: **push** ke branch `main`
- Runner: `ubuntu-latest`
- Steps:
  1. Checkout code
     `main`
     `actions/setup-java@v4`
     `mvn test`
     `mvn jacoco:report`

dan `develop`, serta **pull_request** ke

2. Setup JDK 17 (gunakan)
3. Run tests dengan
4. Generate coverage report dengan
5. Upload artifact dengan nama **"jacoco-report"**

Tuliskan file YAML lengkap!

Jawaban - file .github/workflows/java-unit-test.yml:

```
name: Java Unit Test

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'

      - name: Run tests
        run: mvn test

      - name: Generate coverage report
        run: mvn jacoco:report
```

```
- name: Upload artifact
  uses: actions/upload-artifact@v4
  with:
    name: jacoco-report
    path: target/site/jacoco/
```

BAGIAN D: CODE COVERAGE & INTEGRASI (15 POIN)

Soal 7 (15 Poin)

Analisis Code Coverage:

Berikut adalah hasil coverage report dari sebuah project:

```
text
---------- coverage: platform linux, python 3.11 -----------
Name                        Stmts   Miss  Cover
-----------------------------------------------------------
src/calculator.py              10      0   100%
src/login_system.py            15      2    87%
src/payment.py                 20      8    60%
src/utils.py                    5      1    80%
-----------------------------------------------------------
TOTAL                          50     11    78%
```

Tugas:

1. Hitung **total statement** yang ada di project! (2 poin)
2. Hitung **total statement** yang **tidak tereksekusi** (Miss)! (2 poin)
3. File mana yang memiliki coverage **paling rendah**? Berapa persen? (3 poin)
4. Apa yang harus dilakukan untuk meningkatkan coverage pada file `payment.py`? (4 poin)
5. Menurut Anda, apakah coverage 78% sudah cukup baik? Jelaskan alasannya! (4 poin)

Jawaban:
1. Total statement dalam project: 10 + 15 + 20 + 5 = 50 statement.
2. Total statement yang tidak tereksekusi (Miss): 0 + 2 + 8 + 1 = 11 statement.
3. File dengan coverage paling rendah adalah src/payment.py dengan coverage 60%.
4. Untuk meningkatkan coverage payment.py: identifikasi 8 statement yang belum tereksekusi (umumnya cabang if/else seperti pembayaran gagal, nominal kosong atau negatif, dan metode pembayaran tidak tersedia), lalu tulis unit test khusus untuk setiap cabang tersebut dan jalankan ulang pytest --cov sampai seluruh statement terjalankan.
5. Coverage 78% sudah cukup baik sebagai baseline karena mendekati target umum 80%, tetapi belum ideal. Khususnya pada src/payment.py yang hanya 60% dan menyangkut transaksi keuangan, sebaiknya dinaikkan menjadi minimal 80-90% dengan menambahkan test pada cabang yang belum teruji.

BAGIAN E: STUDI KASUS TERPADU (BONUS 10 POIN)

Soal 8 (Bonus)

Implementasi Lengkap:

Buatlah **project lengkap** dengan ketentuan:

1. **Kode Program:** Buat class `BankAccount` dengan method:
    - `deposit(double amount)` - menambah saldo
    - `withdraw(double amount)` - mengurangi saldo
    - `getBalance()` - mengembalikan saldo
    - Validasi: amount tidak boleh negatif, withdraw tidak boleh melebihi saldo
2. **Unit Test:** Tulis unit test lengkap untuk semua method (minimal 6 test case)
3. **GitHub Actions:** Buat workflow YAML untuk menjalankan test dan generate coverage
4. **Repository:** Upload ke GitHub dan pastikan pipeline berjalan sukses

Output:

- Link repository GitHub
- Screenshot pipeline yang berjalan sukses
- Screenshot coverage report

Jawaban:

1. Class BankAccount:

```
public class BankAccount {

    private double balance;

    public BankAccount() {
        this.balance = 0.0;
    }

    public void deposit(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Amount tidak boleh negatif");
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Amount tidak boleh negatif");
        }
        if (amount > balance) {
            throw new IllegalArgumentException("Saldo tidak mencukupi");
        }
        balance -= amount;
```

```
    }

    public double getBalance() {
        return balance;
    }
}
```

2. Unit test (8 test case):

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class BankAccountTest {

    private final BankAccount acc = new BankAccount();

    @Test
    public void testGetBalanceAwalNol() {
        assertEquals(0.0, acc.getBalance());
    }

    @Test
    public void testDepositMenambahSaldo() {
        acc.deposit(100000);
        acc.deposit(50000);
        assertEquals(150000, acc.getBalance());
    }

    @Test
    public void testDepositNegatif() {
        assertThrows(IllegalArgumentException.class, () -> acc.deposit(-1000));
    }

    @Test
    public void testWithdrawMengurangiSaldo() {
        acc.deposit(100000);
        acc.withdraw(30000);
        assertEquals(70000, acc.getBalance());
    }

    @Test
    public void testWithdrawMelebihiSaldo() {
        acc.deposit(10000);
        assertThrows(IllegalArgumentException.class, () -> acc.withdraw(20000));
    }

    @Test
    public void testWithdrawNegatif() {
        assertThrows(IllegalArgumentException.class, () -> acc.withdraw(-5000));
    }

    @Test
    public void testWithdrawSisaNol() {
        acc.deposit(50000);
        acc.withdraw(50000);
        assertEquals(0.0, acc.getBalance());
    }

    @Test
    public void testKombinasiDepositDanWithdraw() {
```

```
        acc.deposit(200000);
        acc.withdraw(75000);
        acc.deposit(25000);
        assertEquals(150000, acc.getBalance());
    }
}
```

3. Workflow GitHub Actions (.github/workflows/java-unit-test.yml):

```
name: Java Unit Test

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'

      - name: Run tests
        run: mvn test

      - name: Generate coverage report
        run: mvn jacoco:report

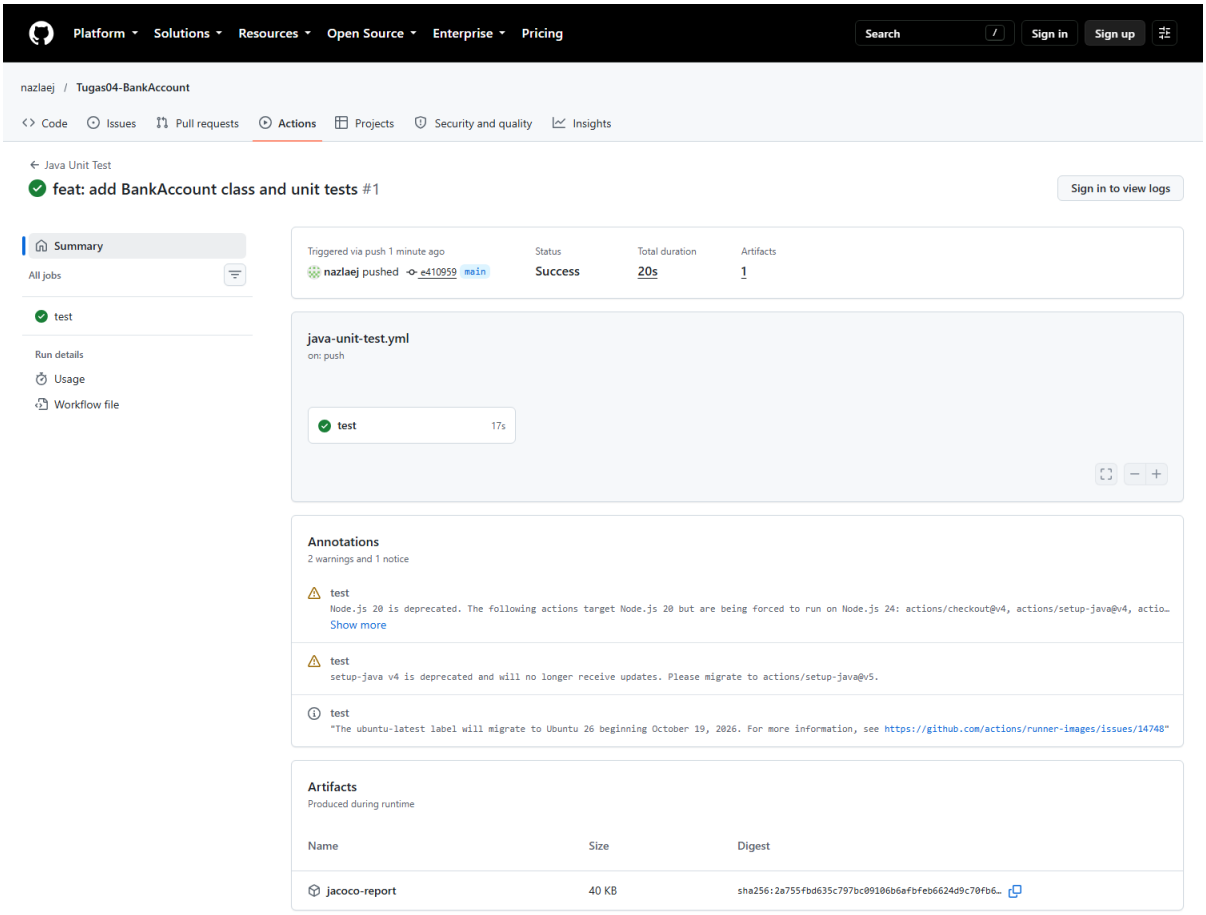
      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

4. File project lengkap (BankAccount.java, BankAccountTest.java, pom.xml, dan workflow) sudah disiapkan pada folder Tugas04_BankAccount. Setelah repository diunggah ke GitHub, isi output berikut pada laporan:

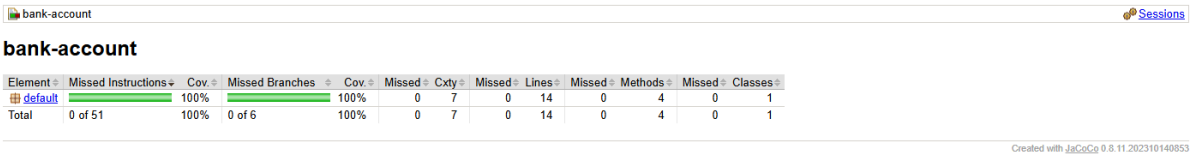
Link repository GitHub: https://github.com/nazlaej/Tugas04-BankAccount

Screenshot pipeline berjalan sukses dan screenshot coverage report dilampirkan pada laporan PDF.

Screenshot pipeline GitHub Actions (status: Success):



Screenshot coverage report (Jacoco - Instructions 100%):



bank-account

Sessions

bank-account

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
default		100%		100%	0	7	0	14	0	4	0	1
Total	0 of 51	100%	0 of 6	100%	0	7	0	14	0	4	0	1

Created with JaCoCo 0.8.11.202310140853

FORMAT PENGUMPULAN

text

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        FORMAT PENGUMPULAN TUGAS                              │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  🗁 Nama File: Tugas04_GitHubActions_[NIM]_[Nama].pdf                        │
│                                                                              │
│  🖹 Format: PDF (diketik, bukan tulis tangan)                                │
│                                                                              │
│  📊 Struktur:                                                                │
```

1. Cover (Nama, NIM, Kelas, Tanggal)
2. Bagian A: Git & GitHub Dasar
3. Bagian B: Unit Testing Framework
4. Bagian C: GitHub Actions
5. Bagian D: Code Coverage & Integrasi
6. Bagian E: Studi Kasus Terpadu (Bonus)
7. Link Repository GitHub

📤 Pengumpulan:

- Upload ke GitHub Repository (folder: Tugas/Tugas04)
- Submit link GitHub ke LMS/Google Classroom

⏰ Deadline: Minggu ke-5, sebelum perkuliahan dimulai

📊 Penilaian:

- Bagian A: 25 poin
- Bagian B: 35 poin
- Bagian C: 25 poin
- Bagian D: 15 poin
- Bagian E: 10 poin (bonus)
- Total: 100 poin (+10 bonus)
